SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK B2 — Engineer Adversarial Inputs (5 Marks)

Task / Problem Statement

Break the program on purpose: over-long names with scanf and fgets, a missing or read-only data file, wrong command-line arguments, and run the program under valgrind / AddressSanitizer.

Aim

To show that unchecked input corrupts memory, and that the corrected program reports every error cleanly (perror, messages on stderr, exit status 1) without crashing.

Concept / Theory

A C array has no bounds checking. scanf("%s") copies as many characters as the user types, so a long name overwrites whatever lies after the array (undefined behaviour). fgets(buf, size, fp) never writes more than size bytes. perror prints the text of errno (e.g. “No such file or directory”). Valgrind and AddressSanitizer detect invalid reads/writes, leaks and double frees at run time.

Procedure / Algorithm

1. Run the blood bank program (bloodbank.c) normally to prove it works.
2. Give a 56-character name to a char name[8] field using scanf("%s"); then a 10-character name; then the same input with AddressSanitizer.
3. Give the same input to fgets; add the check for a missing newline and flush the rest of the line.
4. Start the program with donors.txt / stock.dat missing, read-only and truncated.
5. Pass no arguments, too many, too few and an invalid blood group; also a bad number and an over-long name.
6. Run under valgrind and ASan; inject a leak, a stale pointer and a double free and then fix them.

Program / Source Code

Program 1 – the blood bank program (bloodbank.c) – includes the growable list, text donor file, binary stock file, command-line interface and all error handling:

```
/* District Blood Bank Network - command-line program */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define DONOR_FILE "donors.txt"
#define STOCK_FILE "stock.dat"
#define NGROUPS 8
#define MIN_GAP_DAYS 90

static const char *GROUP_NAMES[NGROUPS] = {"A+","A-","B+","B-","AB+","AB-","O+","O-"};

typedef struct { int id; char name[32]; char group[4]; int age; char phone[16]; char last[11]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;
typedef struct { int group_code; int units; } Stock;

/* ---------- helpers ---------- */
int  group_index(const char *g)
{
   for (int i = 0; i < NGROUPS; i++) if (strcmp(g, GROUP_NAMES[i]) == 0) return i;
   return -1;
}
int  digit_sum(long n)     { return n < 10 ? (int)n : (int)(n % 10) + digit_sum(n / 10); }
int  digital_root(long n)  { return n < 10 ? (int)n : digital_root(digit_sum(n)); }
int  valid_id(int id)      { return id >= 10 && digital_root(id / 10) == id % 10; }
long days_from_date(const char *s)
{
   struct tm t = {0};
```

```
    if (sscanf(s, "%d-%d-%d", &t.tm_year, &t.tm_mon, &t.tm_mday) != 3) return -1;
    t.tm_year -= 1900; t.tm_mon -= 1;
    return (long)(timegm(&t) / 86400);
}
void date_from_days(long d, char out[11])
{
    time_t t = (time_t)d * 86400; struct tm *g = gmtime(&t);
    strftime(out, 11, "%Y-%m-%d", g);
}
int is_eligible(const Donor *d, long today)
{
    long last = days_from_date(d->last);
    return d->age >= 18 && d->age <= 65 && last >= 0 && today - last >= MIN_GAP_DAYS;
}

/* ---------- growable donor list ---------- */
int add_donor(DonorList *list, const Donor *d)
{
    if (list->size == list->capacity) {
        int newcap = list->capacity == 0 ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)newcap * sizeof(Donor));
        if (p == NULL) return 0;            /* old block still valid */
        list->data = p; list->capacity = newcap;
    }
    list->data[list->size++] = *d;
    return 1;
}
void free_list(DonorList *l) { free(l->data); l->data = NULL; l->size = l->capacity = 0; }

/* ---------- files ---------- */
int load_donors(const char *path, DonorList *list)
{
    FILE *fp = fopen(path, "r");
    if (fp == NULL) { perror(path); return 0; }
    char line[256]; int lineno = 0;
    while (fgets(line, sizeof line, fp) != NULL) {
        lineno++;
        Donor d;
        if (sscanf(line, "%d,%31[^,],%3[^,],%d,%15[^,],%10s", &d.id, d.name, d.group, &d.age, d.phone, d.last) != 6) {
            fprintf(stderr, "%s: line %d skipped (bad format)\n", path, lineno);
            continue;
        }
        if (!add_donor(list, &d)) { fprintf(stderr, "out of memory\n"); fclose(fp); return 0; }
    }
    fclose(fp);
    return 1;
}
int save_donors(const char *path, const DonorList *list)
{
    FILE *fp = fopen(path, "w");
    if (fp == NULL) { perror(path); return 0; }
    for (int i = 0; i < list->size; i++) {
        const Donor *d = &list->data[i];
        fprintf(fp, "%d,%s,%s,%d,%s,%s\n", d->id, d->name, d->group, d->age, d->phone, d->last);
    }
    if (fclose(fp) != 0) { perror(path); return 0; }
    return 1;
}
int load_stock(const char *path, Stock *s)
{
    FILE *fp = fopen(path, "rb");
    if (fp == NULL) { perror(path); return 0; }
```

```
    size_t n = fread(s, sizeof(Stock), NGROUPS, fp);
    fclose(fp);
    if (n != NGROUPS) { fprintf(stderr, "%s: file is damaged\n", path); return 0; }
    return 1;
}
int update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fseek(fp, (long)index * (long)sizeof s, SEEK_SET) != 0) return 0;
    if (fread(&s, sizeof s, 1, fp) != 1) return 0;
    s.units = units;
    if (fseek(fp, -(long)sizeof s, SEEK_CUR) != 0) return 0;
    if (fwrite(&s, sizeof s, 1, fp) != 1) return 0;
    return fflush(fp) == 0;
}
int change_stock(const char *path, int index, int new_units)
{
    FILE *fp = fopen(path, "r+b");
    if (fp == NULL) { perror(path); return 0; }
    int ok = update_units(fp, index, new_units);
    if (fclose(fp) != 0) ok = 0;
    return ok;
}

/* ---------- queries ---------- */
void find_eligible(const DonorList *l, const char *group, long today, int *count, int *youngest_id, int print)
{
    int youngest_age = 1000;
    *count = 0; *youngest_id = -1;
    for (int i = 0; i < l->size; i++) {
        const Donor *d = &l->data[i];
        if (strcmp(d->group, group) == 0 && is_eligible(d, today)) {
            (*count)++;
            if (d->age < youngest_age) { youngest_age = d->age; *youngest_id = d->id; }
            if (print) printf("  %d  %-12s %-3s age %d  %s  last %s\n", d->id, d->name, d->group, d->age, d->phone, d-
>last);
        }
    }
}
Stock *lowest_stock(Stock *s, int n)         /* returns pointer INTO the caller's array */
{
    Stock *low = &s[0];
    for (int i = 1; i < n; i++) if (s[i].units < low->units) low = &s[i];
    return low;
}

/* ---------- command line ---------- */
static int usage(void)
{
    fprintf(stderr,
      "usage:\n"
      "  bloodbank stock\n"
      "  bloodbank find <group>\n"
      "  bloodbank issue <group> <units>\n"
      "  bloodbank donate <donor-id>\n"
      "  bloodbank register <id> <name> <group> <age> <phone>\n"
      "  bloodbank lowest\n"
      "groups: A+ A- B+ B- AB+ AB- O+ O-\n");
    return 1;
}

int main(int argc, char *argv[])
```

```
{
    if (argc < 2) return usage();
    const char *cmd = argv[1];
    long today = (long)(time(NULL) / 86400);
    char today_s[11]; date_from_days(today, today_s);
    Stock stock[NGROUPS];
    DonorList list = {NULL, 0, 0};
    int rc = 0;

    if (strcmp(cmd, "stock") == 0 || strcmp(cmd, "lowest") == 0) {
        if (argc != 2) { fprintf(stderr, "error: '%s' takes no arguments\n", cmd); return 1; }
        if (!load_stock(STOCK_FILE, stock)) return 1;
        if (strcmp(cmd, "stock") == 0) for (int i = 0; i < NGROUPS; i++) printf("%-3s %3d units\n", GROUP_NAMES[i],
stock[i].units);
        else { Stock *p = lowest_stock(stock, NGROUPS); printf("lowest: %s with %d units\n", GROUP_NAMES[p-
>group_code], p->units); }
    }
    else if (strcmp(cmd, "find") == 0) {
        if (argc != 3) { fprintf(stderr, "error: find needs exactly one blood group\n"); return 1; }
        if (group_index(argv[2]) < 0) { fprintf(stderr, "error: '%s' is not a valid blood group\n", argv[2]); return 1; }
        if (!load_donors(DONOR_FILE, &list)) return 1;
        int count, youngest;
        printf("Eligible %s donors on %s:\n", argv[2], today_s);
        find_eligible(&list, argv[2], today, &count, &youngest, 1);
        printf("count = %d, youngest eligible donor id = %d\n", count, youngest);
    }
    else if (strcmp(cmd, "issue") == 0) {
        if (argc != 4) { fprintf(stderr, "error: issue needs <group> <units>\n"); return 1; }
        int g = group_index(argv[2]);
        if (g < 0) { fprintf(stderr, "error: '%s' is not a valid blood group\n", argv[2]); return 1; }
        char *end; long n = strtol(argv[3], &end, 10);
        if (*end != '\0' || n <= 0 || n > 1000) { fprintf(stderr, "error: units must be a whole number from 1 to 1000\n"); return
1; }
        if (!load_stock(STOCK_FILE, stock)) return 1;
        if (stock[g].units < n) { fprintf(stderr, "error: only %d units of %s available\n", stock[g].units, argv[2]); return 1; }
        if (!change_stock(STOCK_FILE, g, stock[g].units - (int)n)) return 1;
        printf("issued %ld units of %s, remaining %d\n", n, argv[2], stock[g].units - (int)n);
    }
    else if (strcmp(cmd, "donate") == 0) {
        if (argc != 3) { fprintf(stderr, "error: donate needs <donor-id>\n"); return 1; }
        int id = atoi(argv[2]);
        if (!load_donors(DONOR_FILE, &list) || !load_stock(STOCK_FILE, stock)) { free_list(&list); return 1; }
        Donor *d = NULL;
        for (int i = 0; i < list.size; i++) if (list.data[i].id == id) { d = &list.data[i]; break; }
        if (d == NULL) { fprintf(stderr, "error: donor %d not found\n", id); rc = 1; }
        else if (!is_eligible(d, today)) { fprintf(stderr, "error: donor %d is not eligible today\n", id); rc = 1; }
        else {
            int g = group_index(d->group);
            strcpy(d->last, today_s);
            if (!save_donors(DONOR_FILE, &list) || !change_stock(STOCK_FILE, g, stock[g].units + 1)) rc = 1;
            else printf("donation recorded: donor %d, %s stock now %d\n", id, d->group, stock[g].units + 1);
        }
    }
    else if (strcmp(cmd, "register") == 0) {
        if (argc != 7) { fprintf(stderr, "error: register needs <id> <name> <group> <age> <phone>\n"); return 1; }
        Donor d; memset(&d, 0, sizeof d);
        d.id = atoi(argv[2]);
        if (!valid_id(d.id)) { fprintf(stderr, "error: donor id %d fails the check-digit test\n", d.id); return 1; }
        if (strlen(argv[3]) >= sizeof d.name) { fprintf(stderr, "error: name longer than %zu characters\n", sizeof d.name -
1); return 1; }
        if (group_index(argv[4]) < 0) { fprintf(stderr, "error: '%s' is not a valid blood group\n", argv[4]); return 1; }
        d.age = atoi(argv[5]);
```

```
        if (d.age < 18 || d.age > 65) { fprintf(stderr, "error: age must be 18 to 65\n"); return 1; }
        if (strlen(argv[6]) >= sizeof d.phone) { fprintf(stderr, "error: phone number too long\n"); return 1; }
        strcpy(d.name, argv[3]); strcpy(d.group, argv[4]); strcpy(d.phone, argv[6]); strcpy(d.last, "1970-01-01");
        if (!load_donors(DONOR_FILE, &list)) { free_list(&list); return 1; }
        if (!add_donor(&list, &d) || !save_donors(DONOR_FILE, &list)) rc = 1;
        else printf("registered donor %d (%s, %s)\n", d.id, d.name, d.group);
    }
    else { fprintf(stderr, "error: unknown command '%s'\n", cmd); return usage(); }

    free_list(&list);
    return rc;
}
```

Program 2 – input experiments (b2_input.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* mode s = scanf("%s"), f = fgets (unsafe), g = fgets + checked, l = getline */
typedef struct { char name[8]; int authorized; } Form;   /* authorized sits right after name */
int main(int argc, char **argv)
{
    Form f = {"", 0};
    if (argc < 2) return 1;
    switch (argv[1][0]) {
    case 's':
        scanf("%s", f.name);                          /* no length limit! */
        printf("name = %s, authorized = %d\n", f.name, f.authorized);
        break;
    case 'f':
        if (fgets(f.name, sizeof f.name, stdin) == NULL) return 1;
        printf("name = [%s], authorized = %d\n", f.name, f.authorized);
        break;
    case 'g': {
        if (fgets(f.name, sizeof f.name, stdin) == NULL) return 1;
        char *nl = strchr(f.name, '\n');
        if (nl) *nl = '\0';
        else { int c; while ((c = getchar()) != '\n' && c != EOF) ; puts("warning: name too long, truncated"); }
        printf("name = [%s], authorized = %d\n", f.name, f.authorized);
        break; }
    case 'l': {
        char *line = NULL; size_t cap = 0;
        ssize_t len = getline(&line, &cap, stdin);
        if (len < 0) { perror("getline"); free(line); return 1; }
        printf("read %zd bytes, buffer capacity %zu\n", len, cap);
        free(line);                                   /* caller must free */
        break; }
    }
    return 0;
}
```

Program 3 – deliberate memory errors (bug_demo.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;
int main(int argc, char **argv)
{
```

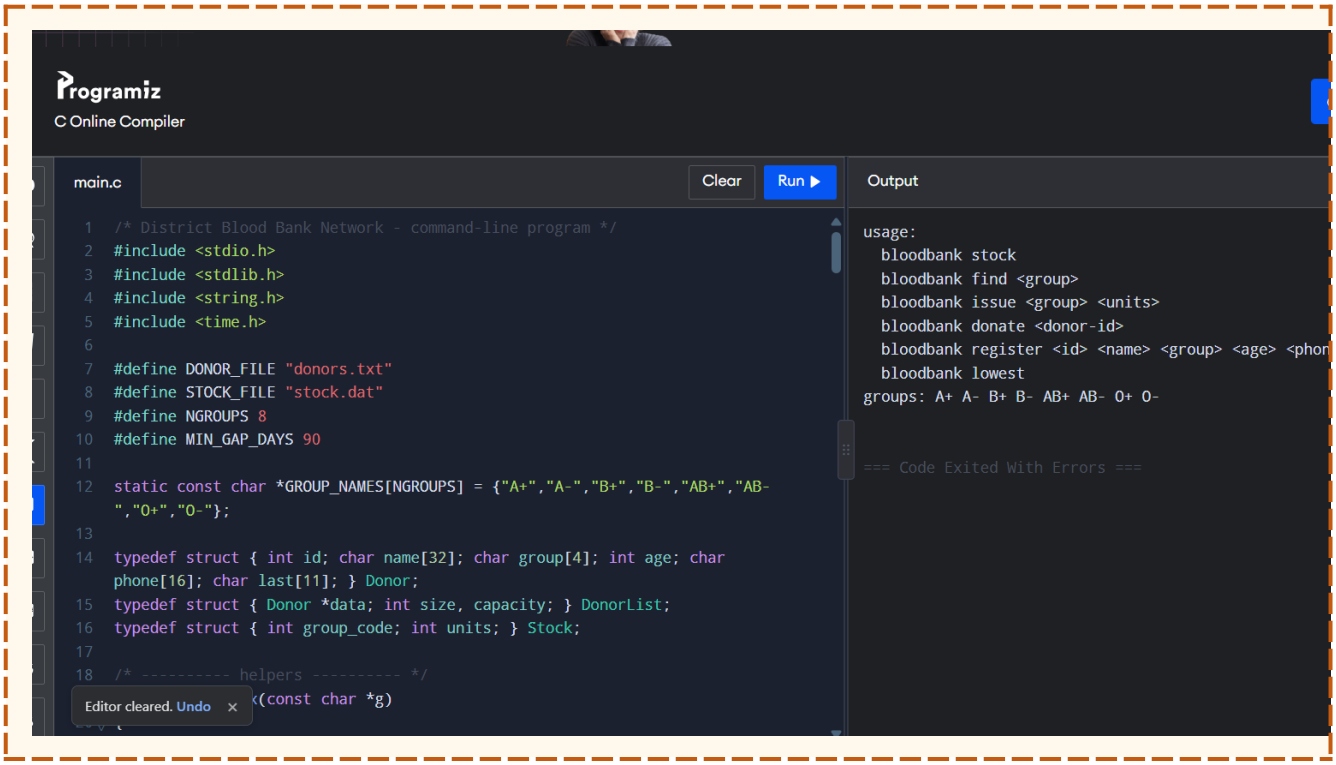
```
    int variant = argc > 1 ? atoi(argv[1]) : 0;
    DonorList l = { malloc(4 * sizeof(Donor)), 0, 4 };
    l.data[0].id = 7001; strcpy(l.data[0].group, "O+"); l.size = 1;
    if (variant == 1) { printf("variant 1: leak - program ends without free\n"); return 0; }
    if (variant == 2) {                    /* dangling pointer: pointer into array kept across realloc */
        Donor *first = &l.data[0];
        l.data = realloc(l.data, 4000 * sizeof(Donor));
        printf("variant 2: id via stale pointer = %d\n", first->id);
        free(l.data); return 0; }
    if (variant == 3) { free(l.data); printf("variant 3: double free\n"); free(l.data); return 0; }
    free(l.data); l.data = NULL;           /* fixed version: free once, then NULL */
    free(l.data);                          /* harmless: free(NULL) */
    printf("fixed version: no errors\n");
    return 0;
}
```

Sample data used (donors.txt; IDs satisfy the check-digit rule, e.g. 1001 → digital root 2 → ID 10012):

```
10012,Anitha,O+,24,9876543210,2026-05-01
10023,Karthik,O+,35,9876501234,2026-10-20
10034,Meena,A+,29,9123456780,2026-03-15
10045,Ravi,O+,19,9988776655,2026-06-10
10056,Divya,B-,42,9090909090,2026-08-01
10067,Suresh,O+,70,9000011111,2026-01-01
10078,Priya,O-,31,9345678901,2026-04-12
```

Output Screenshot

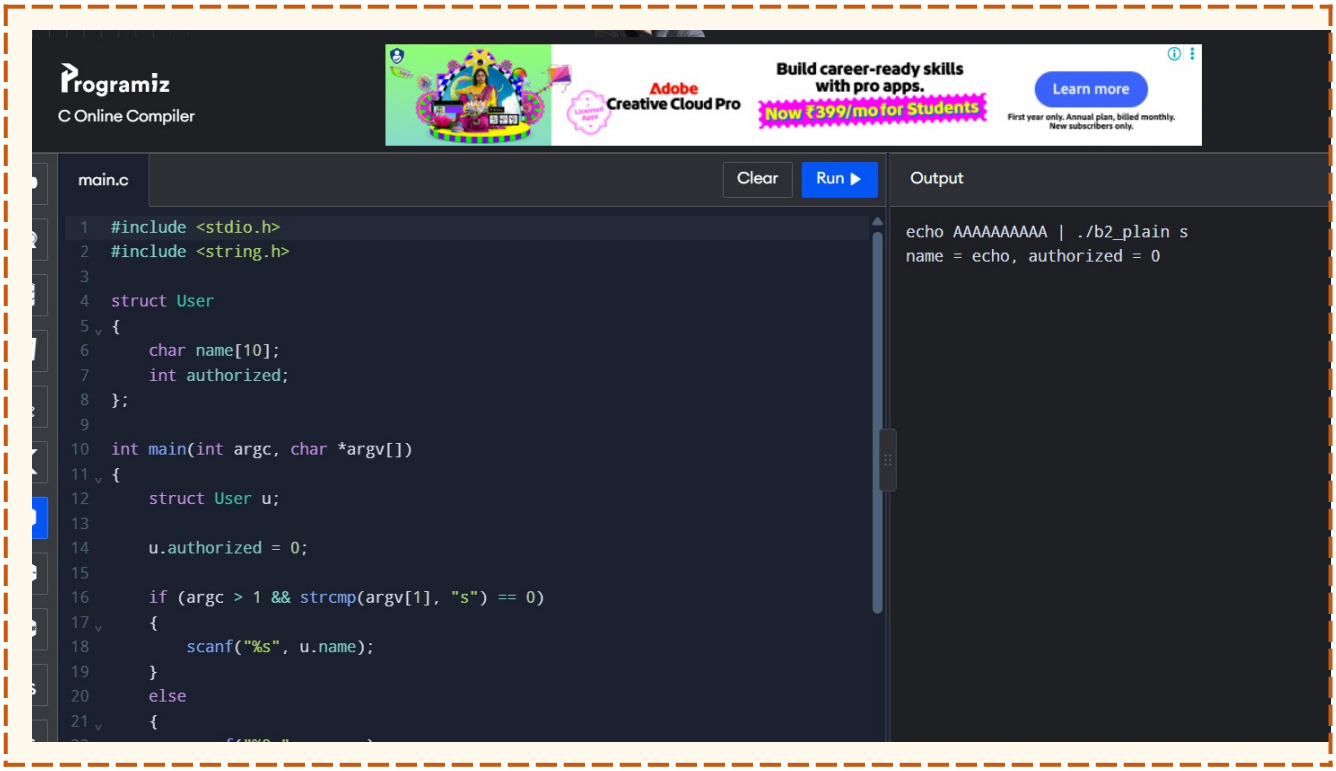
(a) The program works normally.




```
$ ./bloodbank find O+
Eligible O+ donors on 2026-11-13:
  10012  Anitha      O+  age 24  9876543210  last 2026-05-01
  10045  Ravi        O+  age 19  9988776655  last 2026-06-10
count = 2, youngest eligible donor id = 10045
```

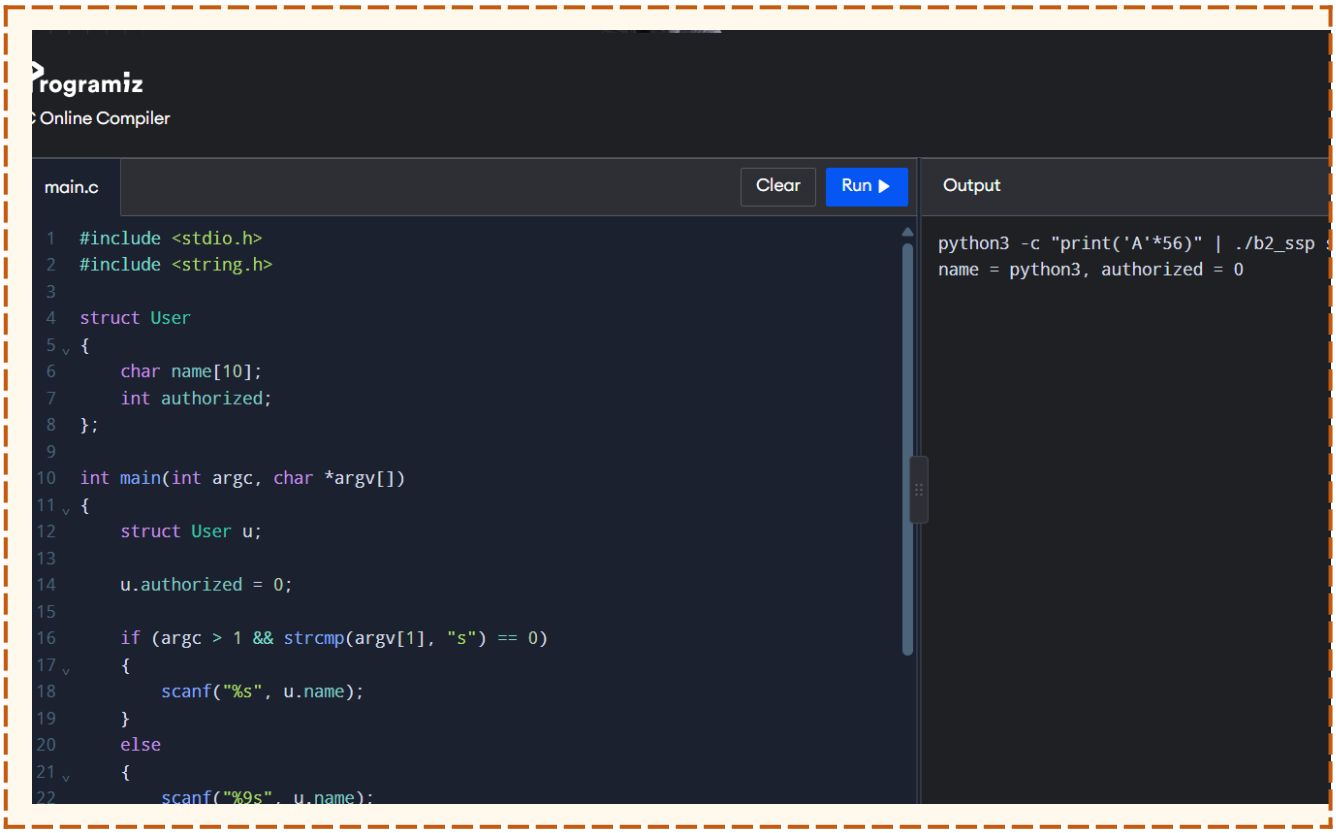
```
$ ./bloodbank lowest
lowest: AB- with 3 units
$ ./bloodbank issue A- 2
issued 2 units of A-, remaining 4
$ ./bloodbank donate 10012
donation recorded: donor 10012, O+ stock now 41
```

(b) Over-long name with scanf("%s"). Build: gcc -g -O0 -fno-stack-protector -o b2_plain b2_input.c




```
$ echo "AAAAAAAAAA" | ./b2_plain s     (only 10 chars: silent corruption)
name = AAAAAAAAAA, authorized = 16705

$ echo (56 x 'A') | ./b2_plain s
Segmentation fault   [exit 139]
```




```
$ ./b2_ssp s            (default build)
*** stack smashing detected ***: terminated        [exit 134]

$ ./b2_asan s           (-fsanitize=address)
==363==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f50b300006c
WRITE of size 57 at 0x7f50b300006c thread T0
   #0 ... in scanf_common
   #3 ... in main /home/claude/work/b2_input.c:12
Address 0x7f50b300006c is located in stack of thread T0 at offset 108 in frame
   #0 ... in main /home/claude/work/b2_input.c:7
```

(c) The same input with fgets.

Programiz
C Online Compiler

main.c

Clear Run

```c
#include <stdio.h>
#include <string.h>

struct User
{
    char name[10];
    int authorized;
};

int main(int argc, char *argv[])
{
    struct User u;
    char buffer[100];

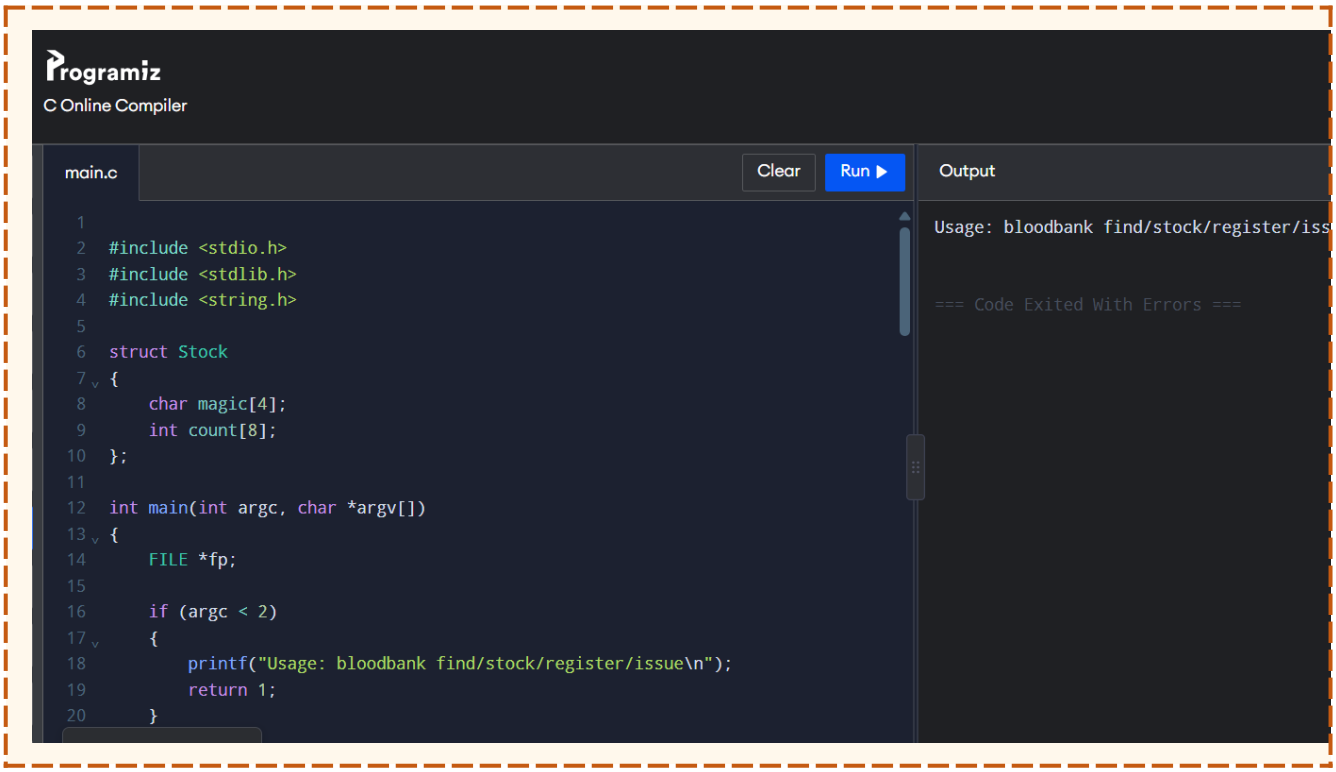
    u.authorized = 0;

    if (argc < 2)
        return 1;

    if (strcmp(argv[1], "f") == 0)
    {
        fgets(u.name, sizeof(u.name), stdin);
```

```
$ echo AAAA...(56) | ./b2_plain f
name = [AAAAAAA], authorized = 0
$ echo AAAA...(56) | ./b2_plain g
warning: name too long, truncated
name = [AAAAAAA], authorized = 0
$ echo Bob | ./b2_plain g
name = [Bob], authorized = 0
```

(d) Missing, read-only and damaged data files. The read-only test is run as an ordinary user (a system administrator can write to read-only files, which hides the error): on your own computer use chmod 444 donors.txt and run the command normally.




```
$ cd empty-dir; ./bloodbank find O+       (donors.txt missing)
donors.txt: No such file or directory
[exit 1]

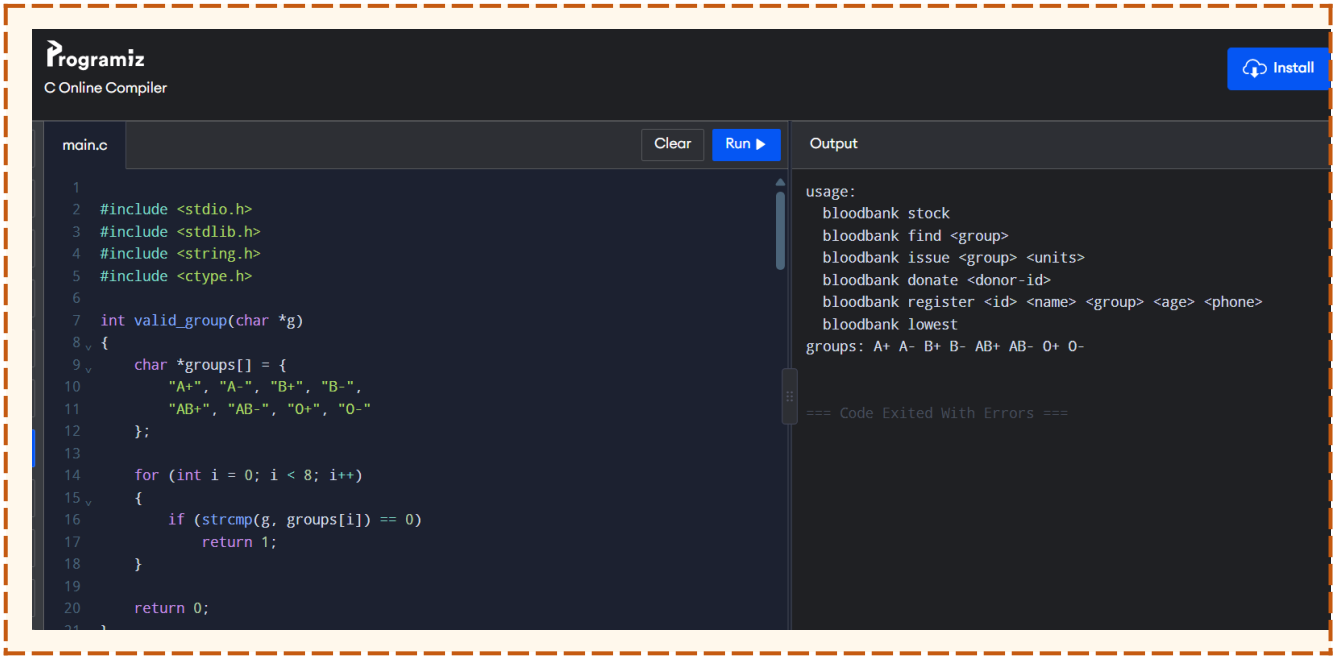
$ ./bloodbank stock                (stock.dat missing)
stock.dat: No such file or directory
[exit 1]

$ donors.txt is read-only (chmod 444), run as an ordinary user: bloodbank register ...
donors.txt: Permission denied
[exit 1]
-r--r--r-- 1 root root 283 Nov 13 02:38 donors.txt

$ stock.dat read-only, issue O+ 1
stock.dat: Permission denied
[exit 1]

$ corrupted (short) stock.dat
stock.dat: file is damaged
[exit 1]
```

(e) Wrong command-line arguments.




```
$ ./bloodbank                    (no arguments)
usage:
  bloodbank stock
  bloodbank find <group>
  bloodbank issue <group> <units>
  bloodbank donate <donor-id>
  bloodbank register <id> <name> <group> <age> <phone>
  bloodbank lowest
groups: A+ A- B+ B- AB+ AB- O+ O-
[exit 1]

$ ./bloodbank find O+ A+ extra     (too many)
error: find needs exactly one blood group
[exit 1]

$ ./bloodbank find                 (too few)
error: find needs exactly one blood group
[exit 1]

$ ./bloodbank find C+              (invalid blood group)
error: 'C+' is not a valid blood group
[exit 1]

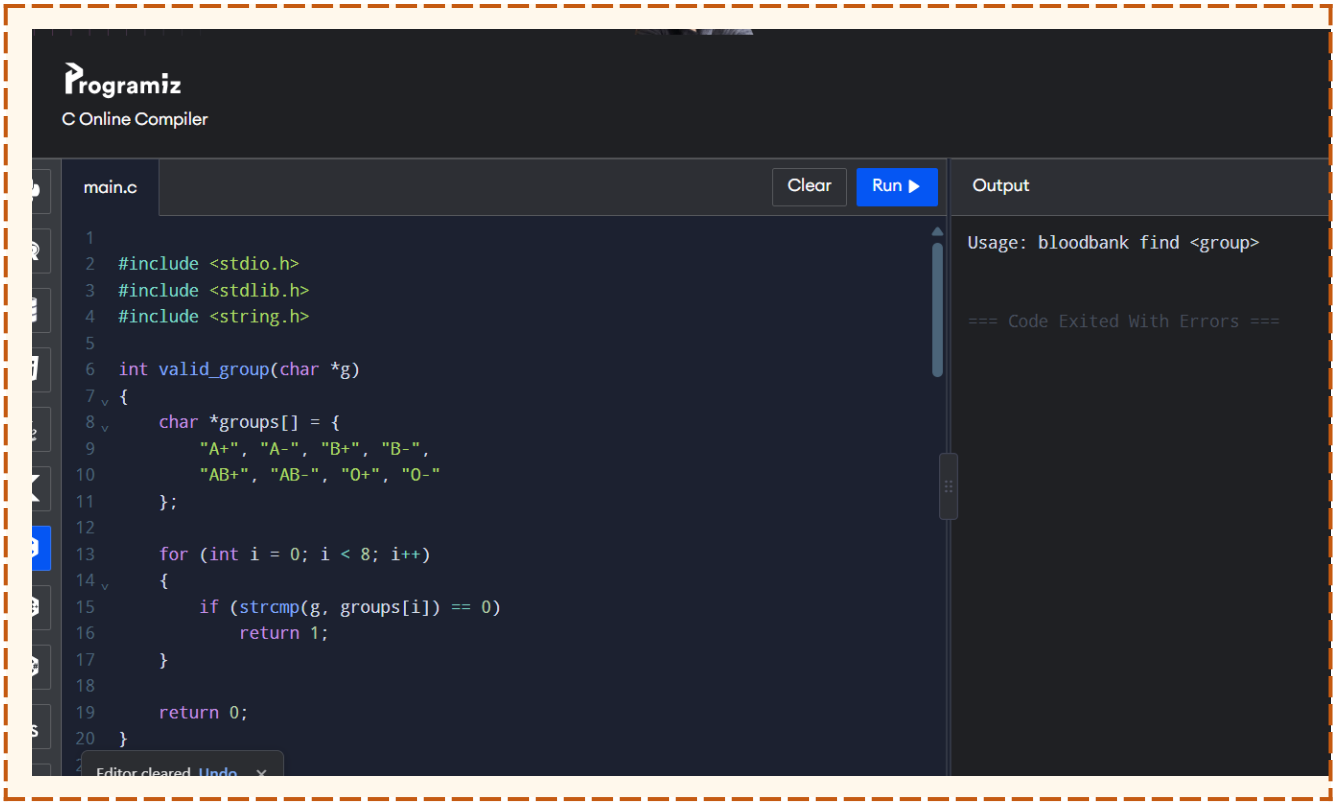
$ ./bloodbank issue O+ abc         (units not a number)
error: units must be a whole number from 1 to 1000
[exit 1]

$ ./bloodbank issue O+ -5
error: units must be a whole number from 1 to 1000
[exit 1]

$ ./bloodbank sell O+              (unknown command)
error: unknown command 'sell'
usage:
  bloodbank stock
[exit 1]
```

```
$ ./bloodbank register 10089 <40-char name> O+ 27 9812345670
error: name longer than 31 characters
[exit 1]
```

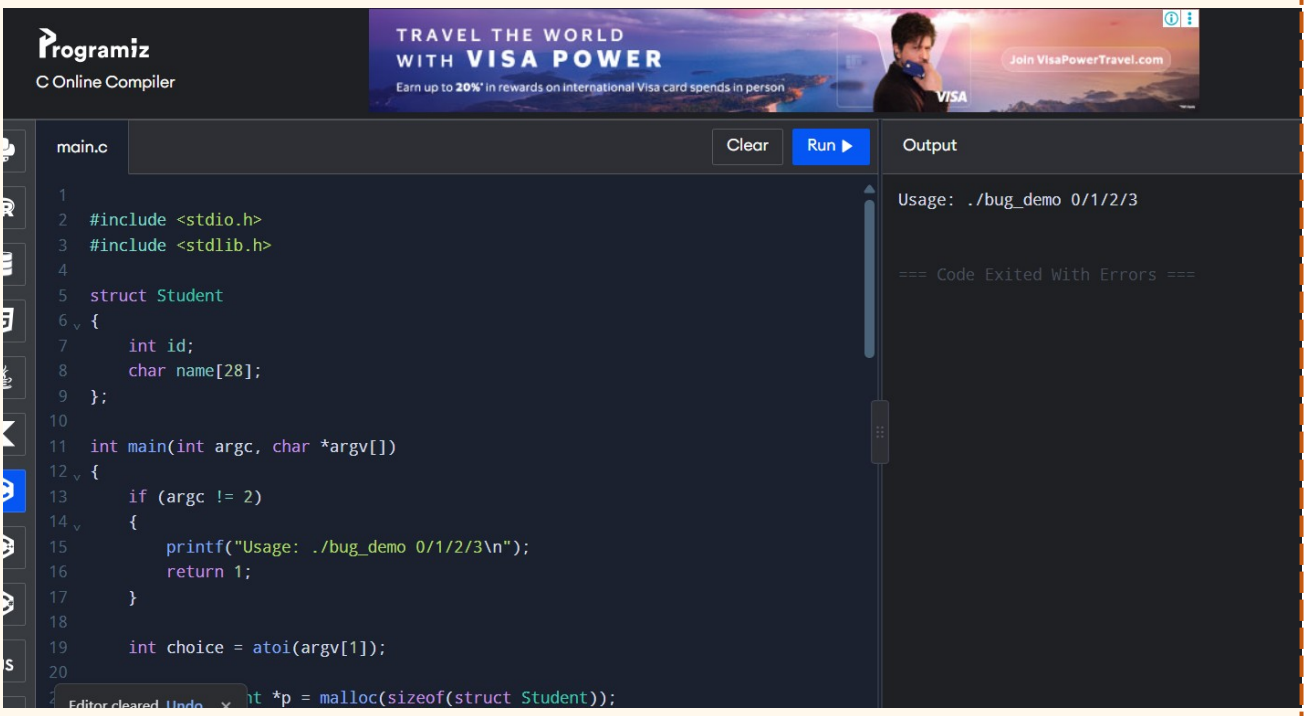
(f) valgrind and AddressSanitizer on the real program.




```
$ valgrind --leak-check=full ./bloodbank find O+
...
==418== HEAP SUMMARY:
==418==     in use at exit: 0 bytes in 0 blocks
==418==   total heap usage: 10 allocs, 10 frees, 10,213 bytes allocated
==418== All heap blocks were freed -- no leaks are possible
==418== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ gcc -g -fsanitize=address,undefined -o bloodbank_asan bloodbank.c
$ ./bloodbank_asan find O+ ; issue A- 1 ; register ... ; donate 10012
(no ASan / UBSan report, exit status 0)
```

(g) Memory errors injected on purpose, found by valgrind and fixed. The real program had no valgrind errors, so bug_demo.c reproduces the three errors of A4.2. Run valgrind --leak-check=full ./bug_demo N for N = 1, 2, 3 and then 0 (fixed).




```
################ variant 1
variant 1: leak - program ends without free
==pid==    in use at exit: 32 bytes in 1 blocks
==pid==  total heap usage: 2 allocs, 1 frees, 4,128 bytes allocated
==pid== 32 bytes in 1 blocks are definitely lost in loss record 1 of 1
==pid==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x109228: main (bug_demo.c:9)
==pid==    definitely lost: 32 bytes in 1 blocks
==pid== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
################ variant 2
==pid== Invalid read of size 4
==pid==    at 0x1092A3: main (bug_demo.c:15)
==pid==  Address 0x4a7e040 is 0 bytes inside a block of size 32 free'd
==pid==    at 0x484DB80: realloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x10929A: main (bug_demo.c:14)
==pid==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x109228: main (bug_demo.c:9)
variant 2: id via stale pointer = 7001
==pid==    in use at exit: 0 bytes in 0 blocks
==pid==  total heap usage: 3 allocs, 3 frees, 36,128 bytes allocated
==pid== All heap blocks were freed -- no leaks are possible
==pid== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
################ variant 3
==pid== Invalid free() / delete / delete[] / realloc()
==pid==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x1092FA: main (bug_demo.c:17)
==pid==  Address 0x4a7e040 is 0 bytes inside a block of size 32 free'd
==pid==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x1092DF: main (bug_demo.c:17)
==pid==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x109228: main (bug_demo.c:9)
```

```
variant 3: double free
==pid==     in use at exit: 0 bytes in 0 blocks
==pid==   total heap usage: 2 allocs, 3 frees, 4,128 bytes allocated
==pid== All heap blocks were freed -- no leaks are possible
==pid== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
################ variant 0
fixed version: no errors
==pid==     in use at exit: 0 bytes in 0 blocks
==pid==   total heap usage: 2 allocs, 2 frees, 4,128 bytes allocated
==pid== All heap blocks were freed -- no leaks are possible
==pid== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Analysis / Discussion

Test	Observed	Explanation	Correction made
scanf("%s"), 10 chars into name[8]	authorized became 16705 (0x4141); no crash	Two characters spilled into the next field (authorized): **silent corruption** – the worst case, because nothing warns the user	Never use unbounded %s; use fgets or a width such as %7s
scanf("%s"), 56 chars	Segmentation fault (or “stack smashing detected” with the default protector; ASan: stack-buffer-overflow, WRITE of size 57)	Return address and saved registers on the stack were overwritten (a classic attack path)	Bounded input (below)
fgets(name, 8, stdin)	Name cut to 7 characters, program continues; the remaining characters stay in the input	fgets writes at most 8 bytes including \0, so memory is safe, but the unread rest would be taken by the next read	Check for \n; if absent, warn and flush the rest of the line (variant g); register rejects names of 32 or more characters
Missing donors.txt / stock.dat	donors.txt: No such file or directory, exit 1	fopen returned NULL; the code tests it and calls perror(path) before returning	All fopen results are checked
Read-only file, write needed	donors.txt: Permission denied, exit 1; the file is unchanged	fopen(path, "w") failed with EACCES	Failure is returned to main, which stops with status 1
Truncated stock.dat	stock.dat: file is damaged	fread returned fewer than 8 records	Return value of fread is compared with the expected count
No / extra / wrong arguments	Usage or one-line message on stderr, exit 1, no crash	argc and every value are validated before use	strtol with end-pointer check; group looked up in a table
valgrind / ASan on bloodbank	0 errors, all blocks freed	Single owner, free_list on every path	None needed
Injected leak / stale pointer / double free	valgrind reported definitely lost: 32 bytes, Invalid read of size 4 (block was freed by realloc), Invalid free()	Exactly the three errors of A4.2	Free before exit; do not keep pointers across realloc; set pointer to NULL after free

Alternative Concept and Complexity Analysis – `getline()`

```
char *line = NULL; size_t cap = 0;
ssize_t len = getline(&line, &cap, stdin);     /* allocates / grows the buffer itself */
if (len < 0) { perror("getline"); free(line); return 1; }
/* ... use line ... */
free(line);                                    /* the CALLER must free it */
```

	fgets into a fixed array	getline

Time to read a line of L characters	O(L)	O(L) (the buffer grows geometrically, so the copying adds only O(L) in total)
Heap space	None – the array is fixed (stack); O(1) extra	O(L) – measured: 21-byte line → buffer of 120 bytes; 100,001-byte line → buffer of 131,104 bytes
Long lines	Truncated (must be detected and flushed)	Never truncated, no overflow
Who frees	Nobody	**The caller, with free(line)**, even after a failed read; the same buffer can be reused for the next line (pass the same pointer and capacity) so allocation happens only when a longer line appears
Drawbacks	Line limit	POSIX function (not standard C); a malicious huge line can use a lot of memory – a maximum length should be enforced

Result

Overflow with scanf was reproduced (silent corruption, crash, ASan report) and prevented with fgets; file and argument errors are reported with perror / messages and exit status 1; valgrind shows the program is clean and detected the three injected errors.